

云南大学信息学院

《算法设计与分析》实验报告



专 业： 计算机科学与技术

年 级： 2024 级

指导教师： 岳昆、吴鑫然

姓 名： ISMAIL ZAYAN AHMED (马子焰)

学 号： 20243060001

2026 年 06 月 06 日

一、实验设置

1.1 实验目的

本实验通过编程实现多种经典算法，并分析不同算法在不同输入规模下的运行效率和性能差异。通过实验验证算法时间复杂度理论，加深对算法设计思想和算法分析方法的理解。

本实验主要完成以下内容：

- (1) 实现冒泡排序、归并排序和快速排序算法；
- (2) 比较不同排序算法在不同输入规模下的比较次数；
- (3) 分析归并排序和快速排序递归过程中子问题规模的变化；
- (4) 实现 0-1 背包问题的蛮力法、动态规划法、贪心法和回溯法；
- (5) 比较不同背包算法在不同输入规模下的运行效率，并分析其时间复杂度和性能特点。

1.2 实验环境

表 1 实验环境配置

项目	配置
CPU	Intel Core i7-12650H (10 核 16 线程, 2.70GHz)
内存	16GB RAM
操作系统	Windows 11
编程语言	C 语言
开发工具	Code::Blocks 20.03
数据处理工具	Microsoft Excel
文档编辑工具	Microsoft Word
绘图工具	Microsoft Excel

二、实验原理

2.1 排序算法

（一）冒泡排序

冒泡排序是一种简单的比较排序算法。算法通过重复比较相邻两个元素，如果顺序错误则交换它们的位置。经过多轮比较后，较大的元素会逐渐移动到序列末尾，因此称为“冒泡排序”。

时间复杂度：

- 最好情况： $O(n)$
- 平均情况： $O(n^2)$
- 最坏情况： $O(n^2)$

伪代码：

BubbleSort(A, n)

for $i \leftarrow 0$ to $n-2$

 for $j \leftarrow 0$ to $n-i-2$

 if $A[j] > A[j+1]$

 交换 $A[j]$ 和 $A[j+1]$

 end if

 end for

end for

（二）归并排序

归并排序采用分治思想。首先将序列不断分成两个子序列，然后分别排序，最后将两个有序子序列合并为一个有序序列。

时间复杂度：

- 最好情况： $O(n \log n)$
- 平均情况： $O(n \log n)$
- 最坏情况： $O(n \log n)$

伪代码：

MergeSort(A, left, right)

if left < right

 mid $\leftarrow$ (left + right) / 2

 MergeSort(A, left, mid)

 MergeSort(A, mid+1, right)

 Merge(A, left, mid, right)

end if

Merge(A, left, mid, right)

 建立左右两个临时数组

 比较两个数组当前元素

 较小元素放入原数组

 重复直到其中一个数组为空

 将剩余元素复制回原数组

（三）快速排序

快速排序也是一种分治算法。算法首先选择一个基准元素（Pivot），然后将序列划分为左右两部分，使左边元素小于基准值，右边元素大于基准值，最后递归完成排序。

时间复杂度：

- 最好情况： $O(n \log n)$
- 平均情况： $O(n \log n)$
- 最坏情况： $O(n^2)$

伪代码:

QuickSort(A, low, high)

if low < high

 pivotIndex $\leftarrow$ Partition(A, low, high)

 QuickSort(A, low, pivotIndex-1)

 QuickSort(A, pivotIndex+1, high)

end if

Partition(A, low, high)

 pivot $\leftarrow$ A[high]

 i $\leftarrow$ low - 1

 for j $\leftarrow$ low to high-1

 if A[j] < pivot

 i $\leftarrow$ i + 1

 交换 A[i] 和 A[j]

 end if

end for

交换 A[i+1] 和 A[high]

```
return i + 1
```

2.2 0-1 背包算法

(1) 蛮力法

蛮力法通过枚举所有可能的物品组合，并计算每种组合的总重量和总价值。在满足背包容量限制的条件下，选择总价值最大的组合作为最优解。该方法能够保证得到最优解，但随着物品数量增加，计算量会迅速增长。

时间复杂度： $O(2^n)$

伪代码：

BruteForceKnapsack (n, Capacity)

bestValue $\leftarrow$ 0

for 每一种物品组合 S

 计算 S 的总重量和总价值

 if 总重量 $\leq$ Capacity

 and 总价值 $>$ bestValue

 bestValue $\leftarrow$ 总价值

 end if

end for

return bestValue

(2) 动态规划法

动态规划法利用状态转移思想求解 0-1 背包问题。算法通过建立二维状态表记录中间结果，避免重复计算，从而提高求解效率。该方法能够保证获得全局最优解，但需要较大的存储空间。

时间复杂度： $O(n \times W)$ 其中 W 为背包容量。

空间复杂度： $O(n \times W)$

伪代码：

DPKnapsack(n, W)

建立二维数组 dp

for $i \leftarrow 1$ to n

 for $w \leftarrow 0$ to W

 if $\text{weight}[i] \leq w$

$dp[i][w] \leftarrow \max ($

$dp[i-1][w],$

$\text{value}[i] + dp[i-1][w - \text{weight}[i]]$

$)$

 else

$dp[i][w] \leftarrow dp[i-1][w]$

```

        end if

    end for

end for

return dp[n][W]

```

(3) 贪心法

贪心法按照单位重量价值（价值/重量）的大小对物品进行排序，每次优先选择当前单位价值最高的物品。如果加入该物品后仍不超过背包容量，则将其放入背包。该方法计算速度快，但对于 0-1 背包问题不能保证获得全局最优解，因此属于近似算法。

时间复杂度： $O(n \log n)$ 主要来自排序过程。

伪代码：

GreedyKnapsack(n , Capacity)

 计算每个物品的价值重量比

 按照价值重量比从大到小排序

 currentWeight $\leftarrow$ 0

 totalValue $\leftarrow$ 0

 for 每个物品 i

 if currentWeight + weight[i]

$\leq \text{Capacity}$

选择物品 i

currentWeight $\leftarrow$

currentWeight + weight[i]

totalValue $\leftarrow$

totalValue + value[i]

end if

end for

return totalValue

(4) 回溯法

回溯法采用深度优先搜索的方式遍历解空间。在搜索过程中，对于每个物品都存在“选择”和“不选择”两种决策，通过递归不断扩展搜索树，最终找到满足容量约束的最优解。

本实验通过递归搜索所有可能情况求解最优价值。

时间复杂度 (最坏情况下): $O(2^n)$

伪代码:

Backtrack(index,

```

        currentWeight,

        currentValue)

if index = n

    更新最优价值

    return

end if

if currentWeight + weight[index]

    ≤ Capacity

    选择当前物品

    Backtrack(

        index+1,

        currentWeight+weight[index],

        currentValue+value[index]

    )

end if

不选择当前物品

Backtrack(

```

```
index+1,  
  
currentWeight,  
  
currentValue  
)
```

在本实验中，分别采用蛮力法、动态规划法、贪心法和回溯法求解 0-1 背包问题。通过比较不同算法在运行时间、最优价值以及空间开销方面的表现，可以分析各算法的优缺点以及适用场景。其中，动态规划法能够获得最优解；贪心法运行速度最快；蛮力法和回溯法适用于较小规模问题。

三、实验数据

3.1 排序算法测试数据

表 2 排序算法比较次数统计表

输入规模	Bubble Sort	Merge Sort	Quick Sort
10	45	21	22
100	4950	539	629
1000	499500	8709	10165
2000	1999000	19392	24467
5000	12497500	55185	66803
10000	49995000	120514	152844

从表 2 可以看出，随着输入规模不断增加，三种排序算法的比较次数都呈增长趋势。其中，冒泡排序的比较次数增长最快。当输入规模达到 10000 时，比较次数达到 49995000 次，远高于另外两种算法。归并排序和快速排序的比较次数明显少于冒泡排序，并且增长速度较慢。实验结果表明，归并排序和快速排序在处理大规模数据时具有更高的效率，其结果与理论时间复杂度 $O(n \log n)$ 基本一致。

表 3 输入等价类测试结果

算法	数据集 1	数据集 2
Bubble Sort	4950	4950
Merge Sort	539	543
Quick Sort	657	739

从表 3 可以看出，在相同输入规模下，不同随机数据集对排序算法的比较次数会产生一定影响。冒泡排序两次实验的比较次数均为 4950 次，说明其比较次数主要与输入规模有关，与数据分布关系较小。

归并排序两次实验结果分别为 539 次和 543 次，变化较小，说明归并排序性能较为稳定。快速排序两次实验结果分别为 657 次和 739 次，差异相对较大。这是因为快速排序的执行过程与基准元素的选择有关，不同数据分布会影响划分结果，从而影响比较次数。

综合来看，归并排序稳定性较好，而快速排序虽然受数据分布影响较明显，但整体效率仍然较高。

3.2 0-1 背包实验数据

物品重量随机生成，范围为 1~100。

物品价值随机生成，范围为 100~1000。

背包容量分别设置为：

10000

100000

1000000

表 4 1000 个物品测试数据（部分）

物品编号	重量	价值
1	91	232
2	26	626
3	97	215
4	41	790
5	26	924
6	36	651
7	60	518
8	87	549
9	61	568
10	38	957

注：由于实验数据总量较大（1000 个物品），表中仅展示前 10 个物品的数据，完整测试数据保存在 Excel 附件（Knapsack1000Data.csv）中。

本实验采用随机生成方式构造测试数据。物品重量范围为 1~100，物品价值范围为 100~1000，共生成 1000 个物品。该数据集用于测试贪心法、动态规划法、回溯法和蛮力法在 0-1 背包问题中的运行效率和求解效果。

四、实验结果与分析

4.1 排序算法实验结果

（1）不同输入规模下比较次数分析

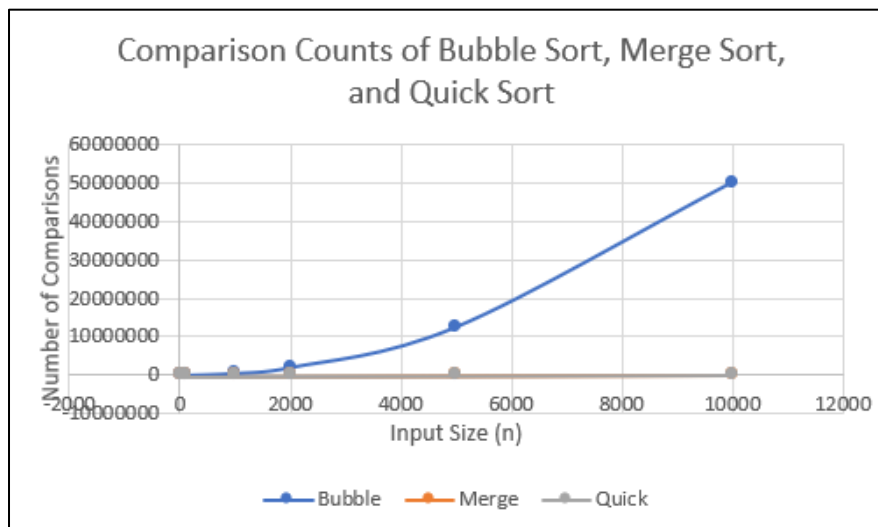


图 1 Bubble Sort、Merge Sort 和 Quick Sort 比较次数对比图

从图 1 可以看出，随着输入规模增加，冒泡排序比较次数增长非常快。当输入规模达到 10000 时，比较次数达到 49995000 次。相比之下，归并排序和快速排序增长速度明显较慢，其实验结果与理论时间复杂度 $O(n \log n)$ 基本一致。

(2) 归并排序与快速排序比较

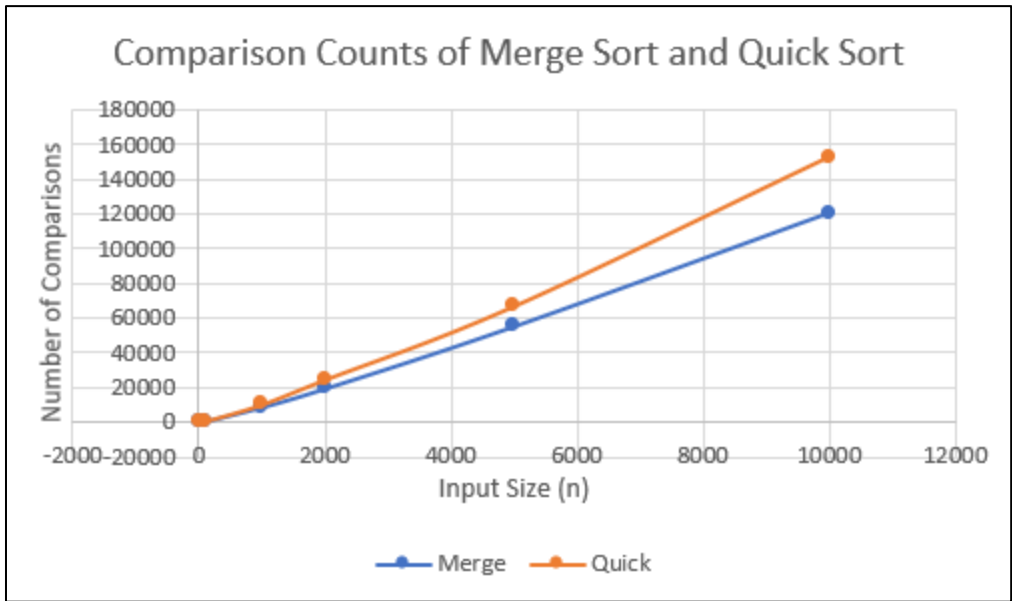


图 2 Merge Sort 与 Quick Sort 比较次数对比图

从图 2 可以看出，归并排序和快速排序均表现出较好的性能。两条曲线变化趋势基本一致，说明两种算法都适合处理大规模数据。

(3) 输入等价类分析

表 5 两组随机数据比较次数统计结果

算法	数据集 1 比较次数	数据集 2 比较次数
冒泡排序	4950	4950
归并排序	539	543
快速排序	657	739

从表 5 可以看出，不同随机数据集对排序算法的比较次数会产生一定影响。冒泡排序在两组数据中的比较次数均为 4950 次，说明其比较次数主要由输入规模决定，与数据分布关系不大，因此结果完全相同。

归并排序在两组数据中的比较次数分别为 539 次和 543 次，差异较小，说明归并排序性能较为稳定，受数据分布影响较弱。快速排序在两组数据中的比较次数分别为 657 次和 739 次，差异相对较大。这是因为快速排序的执行过程与基准元素的选择有关，不同数据分布会影响划分结果，从而影响比较次数。

综合来看，归并排序具有较好的稳定性，而快速排序对输入数据较为敏感，但整体效率仍然较高。

（4）递归子问题规模分析

表 6 归并排序递归子问题规模（部分结果）

递归层次	子问题规模
1	100
2	50
3	25
4	13
5	7
6	4
7	2
8	1

说明：归并排序采用分治策略，每次将问题划分为两个规模更小的子问题，因此子问题规模呈现不断减半的变化趋势。当规模减小到 1 时停止递归，再逐层完成合并操作。实验结果表明，归并排序的递归结构比较规则，递归树较为平衡。

表 7 快速排序递归子问题规模（部分结果）

递归层次	子问题规模
1	100
2	85
3	81
4	11
5	6
6	5
7	3
8	2
9	1

说明：快速排序根据枢轴元素的位置划分子问题。由于每次划分结果不同，因此各子问题规模并不均匀。实验中可以看到，部分子问题规模较大，部分较小，说明快速排序的递归结构受数据分布影响较明显。

4.2 0-1 背包问题实验结果

(1) 贪心算法

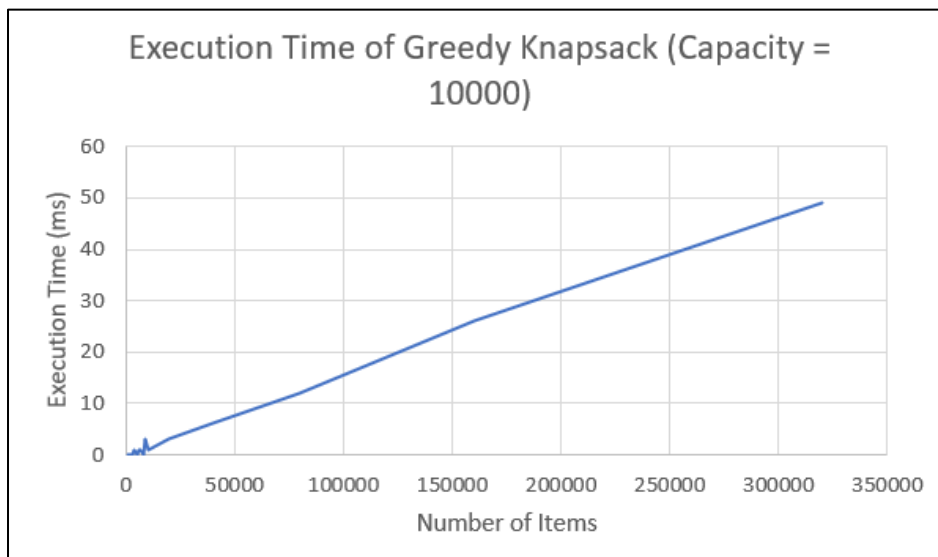


图 3 贪心算法执行时间（容量=10000）

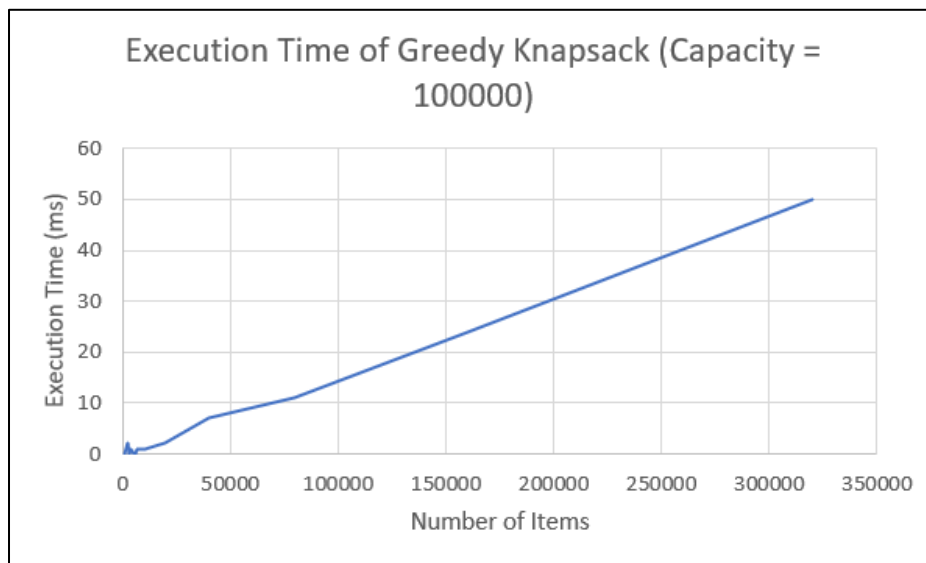


图 4 贪心算法执行时间（容量=100000）

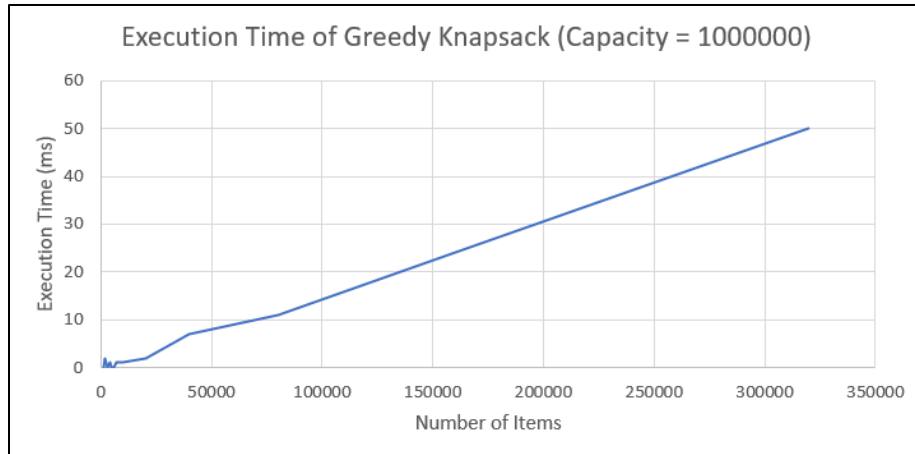


图 5 贪心算法执行时间（容量=1000000）

从图 3 至图 5 可以看出，即使物品数量达到 320000 个，贪心算法仍能够在较短时间内完成计算，说明其具有较高效率。

（2）动态规划算法

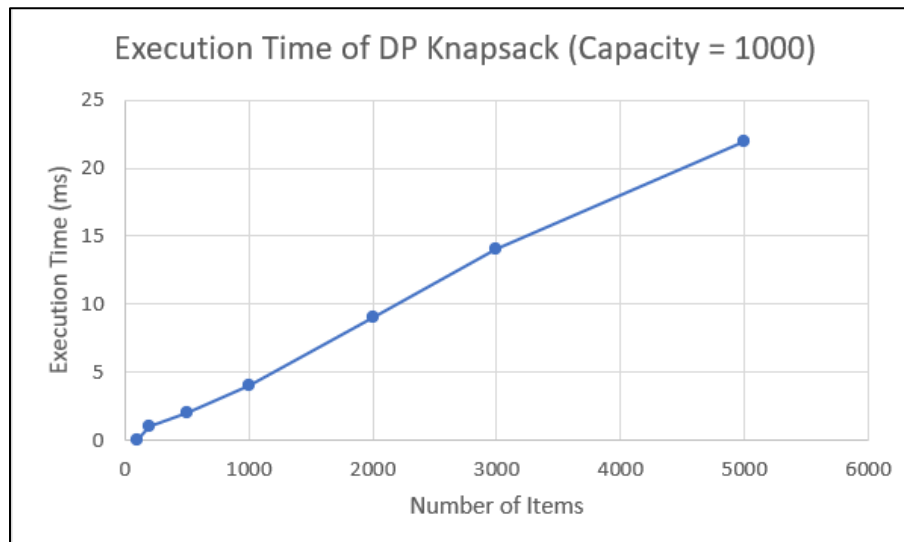


图 6 动态规划执行时间（容量=1000）

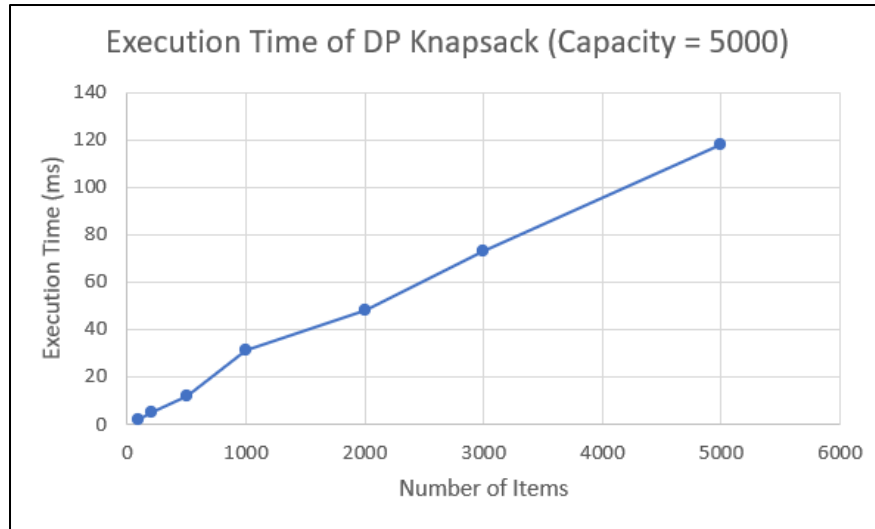


图 7 动态规划执行时间（容量=5000）

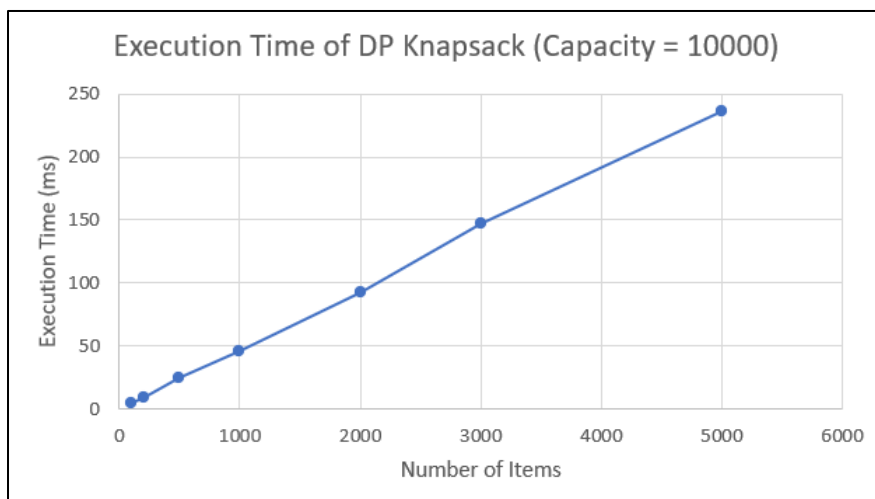


图 8 动态规划执行时间（容量=10000）

实验结果表明，随着物品数量和背包容量增加，动态规划算法运行时间和内存占用均明显增加。

(3) 贪心法与动态规划法比较

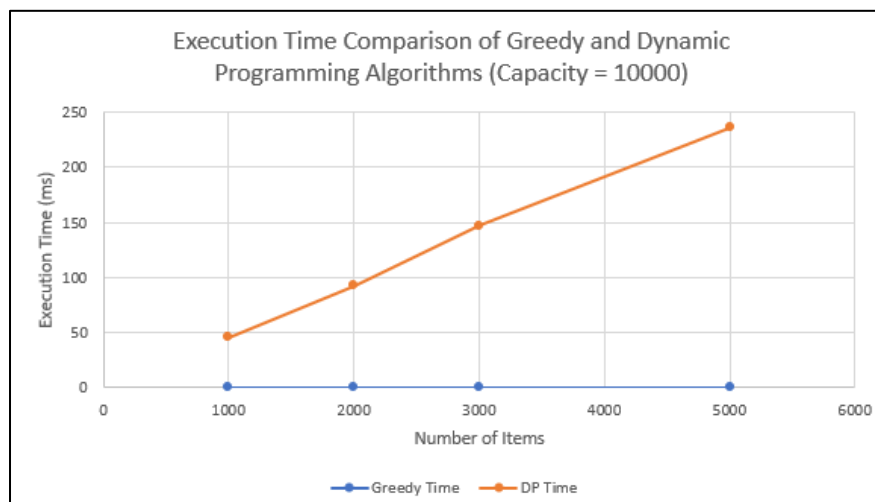


图 9 贪心法与动态规划法执行时间比较

从图 9 可以明显看出，贪心算法执行时间远低于动态规划算法。但是动态规划算法能够保证获得最优解，而贪心算法只能获得近似最优解。

(4) 回溯算法

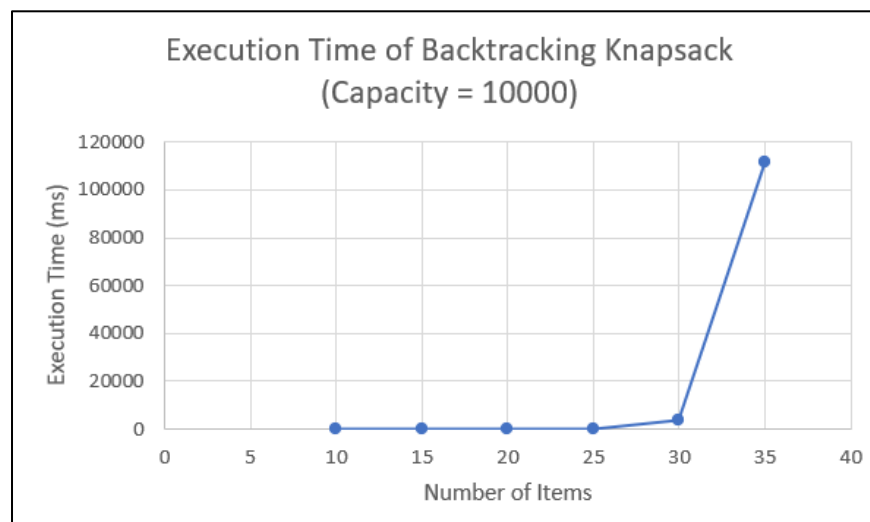


图 10 回溯算法执行时间（容量=10000）

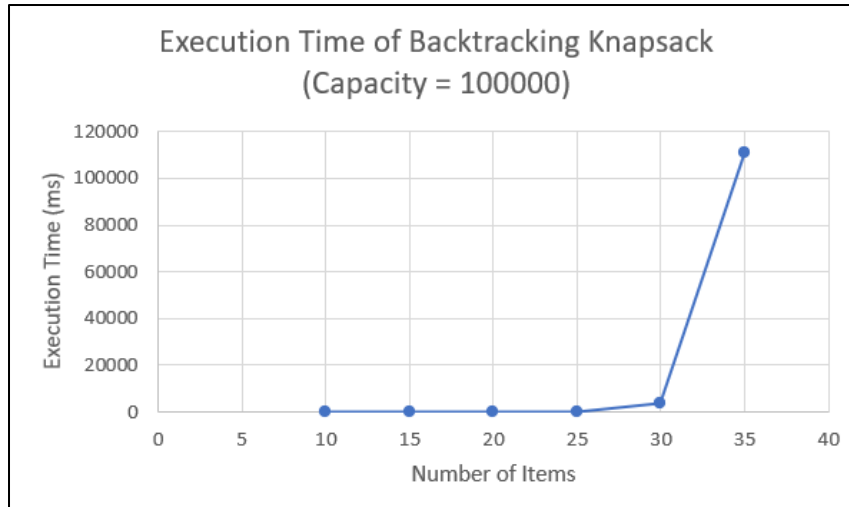


图 11 回溯算法执行时间（容量=100000）

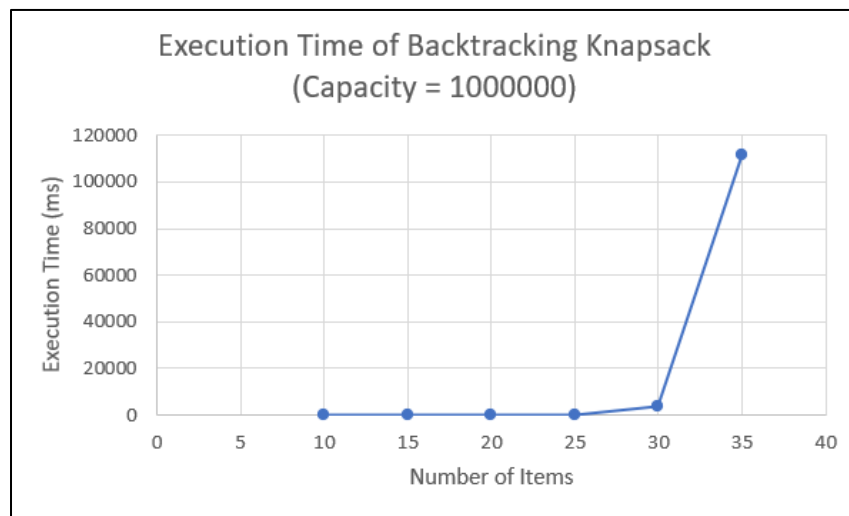


图 12 回溯算法执行时间（容量=1000000）

从实验结果可以看出，当物品数量增加时，回溯算法运行时间迅速增长。例如：25 个物品时约 109ms；35 个物品时已超过 110000ms。

(5) 蛮力算法

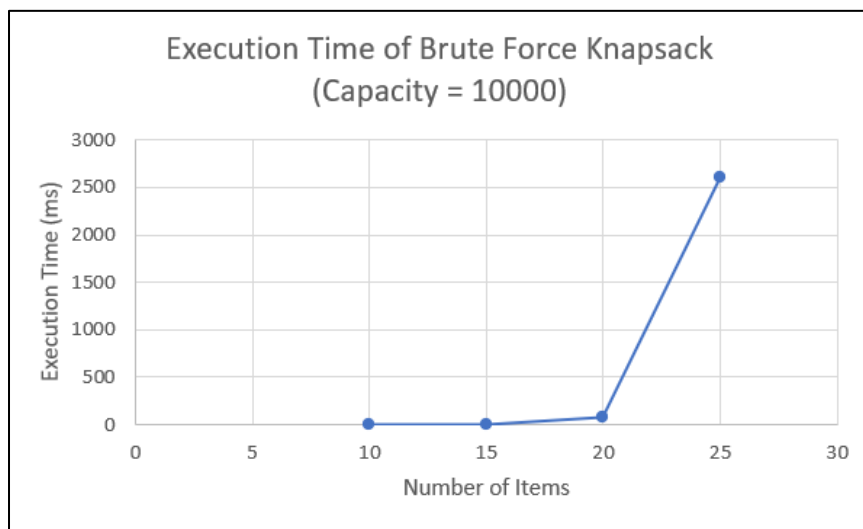


图 13 蛮力法执行时间（容量=10000）

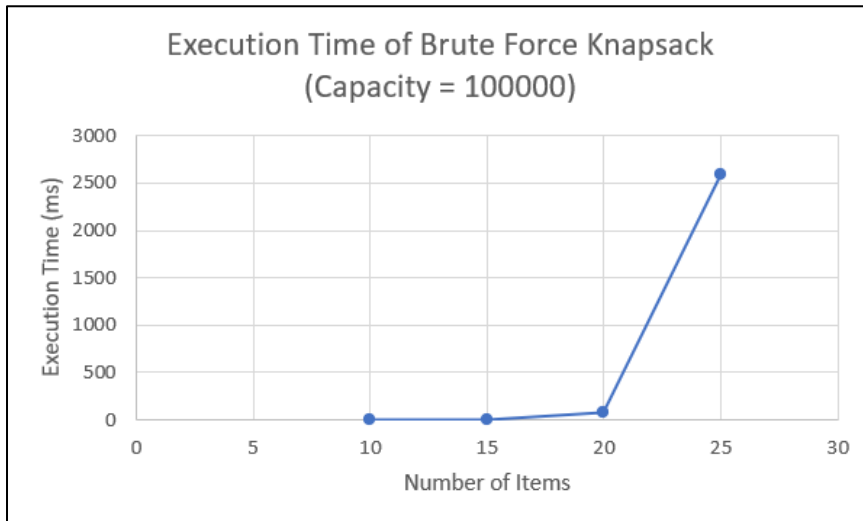


图 14 蛮力法执行时间（容量=100000）

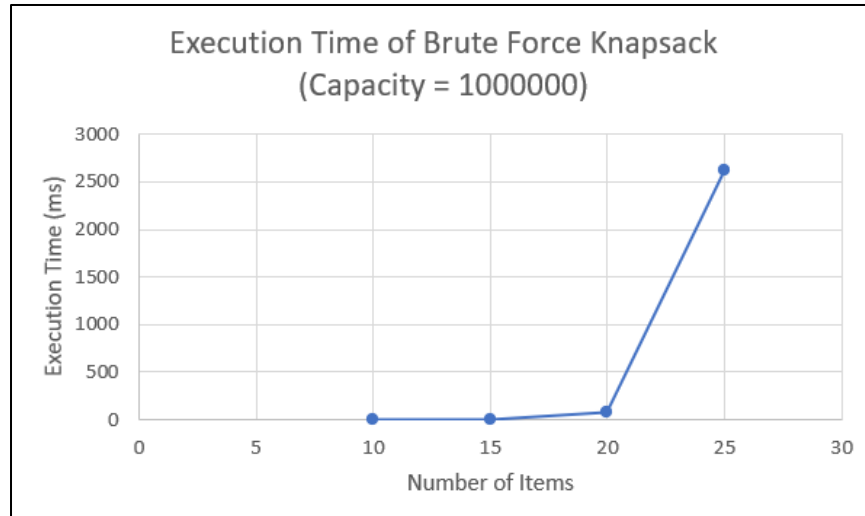


图 15 蛮力法执行时间（容量=1000000）

实验结果表明，蛮力法需要枚举大量可能组合，因此运行效率最低，仅适用于小规模问题。

（6）回溯法与蛮力法比较

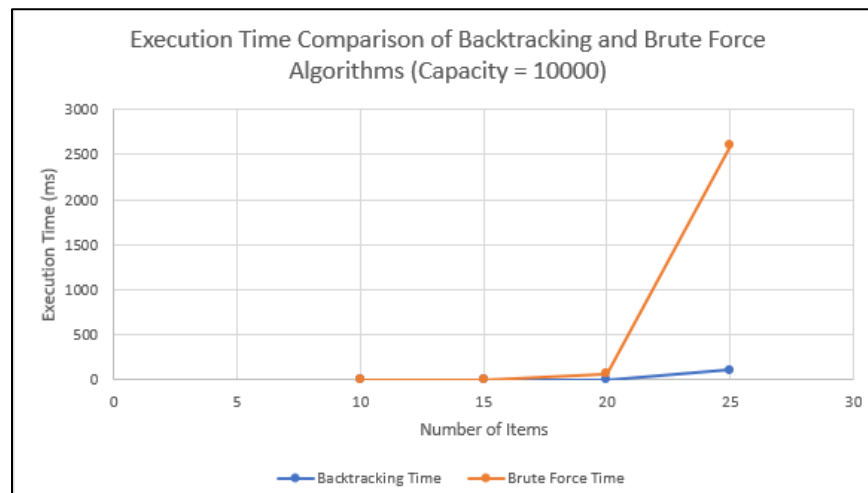


图 16 回溯法与蛮力法执行时间比较

从图 16 可以看出，两种算法都呈现指数级增长趋势。回溯法通过递归搜索解空间，并在超出容量限制时停止继续搜索，因此整体性能通常优于蛮力法。

五、结论

通过本次实验，实现并测试了多种经典排序算法和 0-1 背包问题求解算法。实验结果表明：

- （1）冒泡排序时间复杂度较高，仅适用于小规模数据；
- （2）归并排序和快速排序具有较高效率，更适合处理大规模数据；
- （3）贪心算法执行速度最快，可以处理大量物品；
- （4）动态规划算法能够获得最优解，但需要较大的时间和空间开销；
- （5）回溯法和蛮力法随着问题规模增加，运行时间迅速增长，不适用于大规模问题；
- （6）实验结果与理论复杂度分析基本一致，验证了各种算法的性能特点。

通过本次实验，进一步理解了时间复杂度、空间复杂度以及算法设计思想，提高了利用算法解决实际问题的能力。

六、附录

6.1 GitHub 项目地址

<https://github.com/zayanahmed03/AlgorithmDesignAndAnalysis>

6.2 核心代码

本实验所有源代码、实验数据及统计结果均已上传至 GitHub 仓库，并作为附件提交。下面列出部分核心代码片段。主要代码文件如下：

(1) main.c

实现 Bubble Sort、Merge Sort 和 Quick Sort 的比较次数统计实验。

Bubble Sort 核心代码：

```
for(int i=0;i<n-1;i++)
{
    ...
}
```

Merge Sort 核心代码：

```
if(l < r)
{
    int m = (l+r)/2;

    mergeSort(arr,l,m);
    mergeSort(arr,m+1,r);

    merge(arr,l,m,r);
}
```

(2) RandomDataSet.c

生成随机测试数据，并比较不同随机数据集下各排序算法的性能。

```
generateData(data,n);
```

```
bubbleSort(bubble,n);
```



```
mergeSort(mergeA,0,n-1);
quickSort(quickA,0,n-1);
```

(3) RecursionAnalysis.c

实现归并排序与快速排序递归过程分析，并生成递归规模数据文件。

```
if(low < high)
{
    int pi = partition(arr, low, high);

    quickSort(arr, low, pi - 1);
    quickSort(arr, pi + 1, high);
}
```

(4) GreedyKnapsack.c

实现贪心法求解 0-1 背包问题。

```
qsort(items,n,sizeof(Item),compare);

for(int i=0;i<n;i++)
{
    if(currentWeight + items[i].weight <= capacity)
    {
        currentWeight += items[i].weight;
        totalValue += items[i].value;
    }
}
```

(5) DPKnapsack.c

实现动态规划法求解 0-1 背包问题。

```
if(wt[i-1] <= w)
{
    dp[i][w] = max(
        dp[i-1][w],
        val[i-1] + dp[i-1][w-wt[i-1]]
    );
}
```

(6) BacktrackingKnapsack.c
实现回溯法求解 0-1 背包问题。

```
if(currentWeight + weight[index] <= capacity)
{
    backtrack(
        index+1,
        currentWeight + weight[index],
        currentValue + value[index]
    );
}
```

(7) BruteForceKnapsack.c
实现蛮力法求解 0-1 背包问题。

```
for(long long mask = 0; mask < totalSubsets; mask++)
{
    int currentWeight = 0;
    int currentValue = 0;

    for(int i = 0; i < n; i++)
    {
        if(mask & (1LL << i))
        {
            currentWeight += weight[i];
            currentValue += value[i];
        }
    }

    if(currentWeight <= capacity &&
        currentValue > bestValue)
    {
        bestValue = currentValue;
        bestWeight = currentWeight;
    }
}
```

(8) GenerateKnapsackData.c